

```
class Banner
```

```
attr_accessible :horiz, :link, :visible, :image, :position  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
class Banner < ActiveRecord::Base
```

```
attr_accessible :horiz, :link, :visible, :image  
has_attached_file :image, styles: { vert: '220'
```

```
before_create :assign_position
```

```
protected
```

```
def assign_position
```

```
max = Banner.maximum(:position)
```

```
self.position = max ? max + 1 : 0
```

FULL STACK DEVELOPMENT

REACT - ADVANCED

AULA 04

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON.....	5
SAIBA MAIS	6
O QUE VOCÊ VIU NESTA AULA?	9
REFERÊNCIAS.....	10

EMSE

O QUE VEM POR AÍ?

Prepare-se para transformar a maneira como você desenvolve formulários em React! Nesta aula, aprenderemos a construir um formulário de cadastro sofisticado e funcional, integrando validação de dados e comunicação com um backend. Imagine criar um sistema de cadastro onde os usuários podem inserir suas informações, receber feedback imediato sobre qualquer erro de entrada, e enviar seus dados de maneira segura e eficiente para um servidor. Este é o poder combinado do React, Formik, Yup e axios!

Iniciaremos com uma base sólida, criando um formulário básico com componentes fundamentais como Container, GlobalStyles, Heading, Fieldset, Label, Input, Button e Form. Essa estrutura inicial permitirá que você entenda a organização de um formulário em React, separando cada parte do formulário em componentes reutilizáveis para facilitar a manutenção e a escalabilidade do seu código.

A seguir, mergulharemos na validação de formulários usando Formik e Yup. Você aprenderá a configurar esquemas de validação detalhados que garantem que os dados inseridos pelos usuários estejam corretos antes mesmo de serem enviados. Veremos como fornecer feedback imediato e útil aos usuários, destacando campos obrigatórios, verificando o formato do e-mail e garantindo que as senhas coincidam. A integração do Formik com o Yup tornará o processo de validação simples e eficiente, eliminando a necessidade de validações manuais e reduzindo a chance de erros.

Com a validação no lugar, avançaremos para a integração do formulário com um backend. Utilizando axios, você aprenderá a enviar os dados do formulário para um servidor, lidando com as respostas de sucesso e erro. Exploraremos a criação de funções que manipulam o envio de dados e discutiremos como gerenciar o estado do formulário após a submissão, garantindo que os dados sensíveis sejam tratados de maneira segura e que o usuário receba feedback adequado.

Além disso, refinaremos nossa aplicação para garantir que o código esteja organizado e siga as melhores práticas. Você verá como estruturar seu projeto de forma que ele seja fácil de entender e manter, permitindo futuras expansões e modificações sem complicações. Discutiremos também as vantagens de usar

bibliotecas específicas para diferentes tarefas, destacando a importância de escolher as ferramentas certas para cada parte do desenvolvimento.

Ao final desta aula, você terá a capacidade de criar formulários robustos e dinâmicos, integrando validação avançada e comunicação com backend de maneira eficiente. Essa habilidade não só melhorará suas aplicações React, mas também aumentará sua capacidade de desenvolver interfaces de usuário modernas e responsivas, prontos para enfrentar desafios reais no desenvolvimento web.



HANDS ON

Nesta aula, você terá a oportunidade de acompanhar e implementar, passo a passo, um projeto prático. Nosso objetivo é desenvolver um formulário de cadastro em React, integrando funcionalidades essenciais como validação de dados e comunicação com um backend. Cada vídeo está cuidadosamente estruturado para te guiar pelos conceitos e práticas necessárias para construir uma aplicação robusta e eficiente.

Na primeira videoaula, começaremos configurando a estrutura inicial do nosso formulário. Utilizaremos componentes como Fieldset, Label, Input, Button e Form e introduziremos a biblioteca Formik para gerenciar o estado do formulário e suas submissões. Você aprenderá a definir os valores iniciais do formulário e a lidar com a submissão dos dados, visualizando como o Formik simplifica o gerenciamento de formulários em React.

No segundo vídeo, avançaremos para a validação do formulário utilizando a biblioteca Yup. Aprenderemos a definir um esquema de validação que garantirá que todos os dados inseridos pelos usuários sejam corretos e completos antes de serem enviados. Implementaremos feedback de erros diretamente nos campos do formulário, melhorando a experiência do usuário e a qualidade dos dados recebidos.

Na terceira e última videoaula, adicionaremos a funcionalidade de envio de dados para um backend utilizando axios. Aprenderemos a configurar e enviar requisições HTTP POST para criar usuários, tratar as respostas do servidor, e gerenciar o estado do formulário após o envio. Este passo crucial permitirá que você conecte o frontend ao backend de maneira eficiente e segura.

SAIBA MAIS

Mergulhando fundo no Formik

Nesta aula, aprenderemos sobre uma ferramenta incrível que está mudando a forma como lidamos com formulários em React: o Formik. Se você já desenvolveu aplicações React, sabe que formular e gerenciar formulários pode ser uma tarefa bastante desafiadora. Vamos explorar o motivo do Formik ser tão bem recebido pela comunidade de pessoas desenvolvedoras e aprender como ele pode simplificar nossas vidas.

O que é o Formik?

Formik é uma biblioteca para React que nos ajuda a gerenciar formulários de maneira eficiente e elegante. Ele cuida do estado do formulário, validação, manipulação de envios, e mais. Com o Formik, podemos criar formulários complexos sem nos perdemos em um mar de códigos.

A comunidade de desenvolvimento abraçou o Formik por várias razões:

- **Simplicidade:** abstrai muitas das tarefas repetitivas e tediosas envolvidas na criação e gerenciamento de formulários.
- **Flexibilidade:** oferece uma API poderosa que pode ser adaptada para atender às necessidades de formulários simples e complexos.
- **Integração com Yup:** Yup é uma biblioteca de validação que se integra perfeitamente com o Formik, permitindo validações complexas de forma simples e declarativa.
- **Manutenção de estado:** Formik gerencia o estado do formulário, facilitando o rastreamento de valores, erros e estados de toque (touched state).
- **Comunidade ativa:** como é amplamente utilizado, há uma grande quantidade de recursos e suporte disponíveis, desde tutoriais a fóruns de discussão.

Problemas que o Formik resolve

Vamos pensar em alguns dos desafios que enfrentamos ao lidar com formulários diretamente em React, sem o auxílio de uma biblioteca como o Formik:

- Gerenciamento de estado: cada campo do formulário precisa gerenciar seu próprio estado. Isso pode levar a muitos códigos repetitivos e difíceis de manter.
- Validação: validar os dados de um formulário pode se tornar rapidamente complicado, especialmente quando há muitas regras de validação.
- Envio de dados: gerenciar o processo de envio de dados e lidar com feedback de sucesso ou erro pode se tornar um pesadelo logístico.
- Manutenção: manter formulários complexos e garantir que todos os aspectos do estado do formulário estejam sincronizados pode ser extremamente desafiador.

Vamos ver um exemplo simples. Imagine que estamos criando um formulário de cadastro sem Formik. Teríamos que:

- Criar estados individuais para cada campo do formulário;
- Escrever funções para manipular as mudanças de estado para cada campo;
- Implementar lógica de validação manualmente para cada campo;
- Lidar com o processo de envio dos dados e gerenciamento de erros.

Tudo isso pode rapidamente levar a um código complicado e difícil de gerenciar, especialmente à medida que o formulário cresce em complexidade. Com o Formik, podemos simplificar radicalmente essa abordagem. Vamos ver como isso funciona com um exemplo prático:

```
import { Formik, Form, Field, ErrorMessage } from 'formik';
import * as Yup from 'yup';

const SignupForm = () => (
  <Formik
    initialValues={{ name: '', email: '', password: '',
confirmPassword: '' }}
    validationSchema={Yup.object({
      name: Yup.string().required('O nome é obrigatório'),
      email: Yup.string().email('Por favor, informe um e-
mail válido').required('Por favor, informe o seu email'),
      password: Yup.string().min(8, 'A senha deve possuir
pelo menos 8 caracteres').required('A senha é obrigatória'),
      confirmPassword:
Yup.string().oneOf([Yup.ref('password'), null], 'As senhas devem
ser iguais!').required('Confirme sua senha!'),
    })}
    onSubmit={values, { setSubmitting, resetForm }} => {
      // Aqui enviamos os dados para o servidor
      console.log(values);
    }
  />
)
```

```

        // Resetar o formulário após o envio
        resetForm();
        setSubmitting(false);
    }}
>
    ({ isSubmitting }) => (
      <Form>
        <div>
          <label htmlFor="name">Nome</label>
          <Field name="name" type="text" />
          <ErrorMessage name="name" component="div" />
        </div>
        <div>
          <label htmlFor="email">E-mail</label>
          <Field name="email" type="email" />
          <ErrorMessage name="email" component="div" />
        </div>
        <div>
          <label htmlFor="password">Senha</label>
          <Field name="password" type="password" />
          <ErrorMessage name="password" component="div" />
        </div>
        <div>
          <label htmlFor="confirmPassword">Confirme sua
senha</label>
          <Field name="confirmPassword" type="password" />
          <ErrorMessage
component="div" />
            name="confirmPassword"
          />
        </div>
        <button type="submit" disabled={isSubmitting}>
          Enviar
        </button>
      </Form>
    )}
  </Formik>
);

export default SignupForm;

```

Código-fonte 1 – Exemplo de código-fonte React
 Fonte: Elaborado pelo autor (2024)

Ou seja, o Formik resolve muitos dos problemas associados ao gerenciamento de formulários em React, fornecendo uma solução completa e fácil de usar para inicializar estados, validar entradas e manipular submissões. Isso não só simplifica nosso código, mas também torna nossos formulários mais robustos e fáceis de manter.

O QUE VOCÊ VIU NESTA AULA?

Nesta aula, mergulhamos fundo no uso do Formik, uma poderosa biblioteca para gerenciar formulários em React. Aprendemos como o Formik pode simplificar a criação e manutenção de formulários complexos, abordando os desafios comuns que enfrentamos ao usar apenas o React. O Formik foi bem recebido pela comunidade de desenvolvimento devido à sua simplicidade, flexibilidade e integração perfeita com a biblioteca Yup para validação de dados.

Exploramos como o Formik facilita o gerenciamento de estado dos formulários, abstraindo tarefas repetitivas e tediosas. Com ele, podemos definir estados iniciais, gerenciar mudanças nos campos do formulário e manipular a submissão de dados de maneira centralizada e eficiente. Isso resolve problemas comuns de gerenciamento de estado, validação e envio de dados que, de outra forma, exigiriam muito código manual e difícil de manter.

Também vimos como integrar o Yup ao Formik para validar nossos formulários. Com o Yup, definimos esquemas de validação que garantem a integridade dos dados inseridos pelos usuários. Isso inclui verificar se todos os campos obrigatórios estão preenchidos, se os formatos de dados são válidos e se as senhas coincidem. A combinação do Formik com o Yup nos permite fornecer feedback instantâneo e útil aos usuários, melhorando a experiência do usuário e a qualidade dos dados coletados.

Além disso, aprendemos a estilizar nossos componentes de formulário com Formik, utilizando o componente Field para vincular os campos ao estado gerenciado e o componente ErrorMessage para exibir mensagens de erro de forma elegante. Essa abordagem não só torna nosso código mais limpo e organizado, mas também facilita a manutenção e a escalabilidade de nossos formulários.

Por fim, abordamos a integração do formulário com um backend usando a biblioteca axios. Aprendemos a enviar os dados do formulário para um servidor, lidar com as respostas e gerenciar o estado do formulário após a submissão. Isso é crucial para criar aplicações que não apenas coletam dados de forma eficiente, mas também os processam e os armazenam de maneira segura e confiável.

REFERÊNCIAS

FORMIK. **Formik: Build forms in React, without the tears**. [s.d.]. Disponível em: <https://formik.org/>. Acesso em: 26 jun. 2024.

FORMIK. **Overview**. [s.d.]. Disponível em: <https://formik.org/docs/overview>. Acesso em: 26 jun. 2024.

FORMIK. **Tutorial**. [s.d.]. Disponível em: <https://formik.org/docs/tutorial>. Acesso em: 26 jun. 2024.

GITHUB. **API Reference**. 2024. Disponível em: <https://github.com/jquense/yup#api>. Acesso em: 26 jun. 2024.

GITHUB. **Getting Started**. 2024. Disponível em: <https://github.com/jquense/yup>. Acesso em: 26 jun. 2024.

PALAVRAS-CHAVE

Formik, Yup, validação de formulários, React, gerenciamento de estado, integração backend.

EMSE

POS TECH